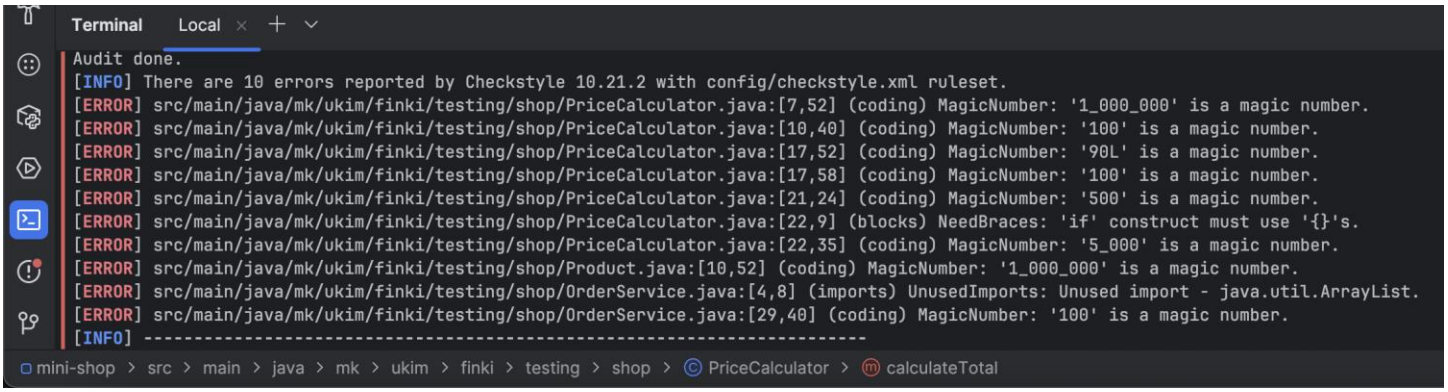
Студент: Христина Ѓорѓиевска 233215

ИЗВЕШТАЈ ЗА ЛАБОРАТОРИСКА ВЕЖБА 1

1. СТАТИЧКА АНАЛИЗА

Откако направив статичка анализа на кодот со командата **./mvnw -Pstatic-analysis validate**, во кодот беа пронајдени грешки во три класи: **Product.java**, **PriceCalculator.java** и **OrderService.java**.



1. Грешки во PriceCalculator.java

- Најголем дел од грешките беа во оваа класа, односно 6 грешки кои беа од типот **MagicNumber** и една од типот **NeedBraces** (if кој нема {} околу условот).

- **MagicNumber** укажува дека во кодот се користат броеви напишани директно во кодот, без име што опишува што всушност значат. Кога се напишани на таков начин, тешко е да се разбере што претставуваат. Уште еден проблем кој може да се јави е, ако иста вредност се користи на повеќе места, при промена многу лесно може да се заборави да се смени на сите места. Затоа, **за такви вредности се препорачува да се заменат со константа со име што точно ја опишува**.

- **NeedBraces** укажува дека во кодот имаме if чие тело не е ставено во {}. Без заградите, само првата линија по условот припаѓа на if-от. Затоа, **за if услови се препорачува телото да биде во {}**.

- **Поправка**:

1. За MagicNumber, броевите ги заменив со статички променливи чие име ја опишува вредноста
2. За NeedBraces, додадов {} околу телото на if условот.

КОДОТ ПРЕД ПРОМЕНИТЕ

```java
public class PriceCalculator {

    public int calculateTotal(int unitPriceCents, int quantity, boolean premiumCustomer) {
        if (unitPriceCents < 1 || unitPriceCents > 1_000_000) {
            throw new IllegalArgumentException("Unit price must be between 1 and 1000000 cents");
        }
        if (quantity < 1 || quantity > 100) {
            throw new IllegalArgumentException("Quantity must be between 1 and 100");
        }

        int subtotal = unitPriceCents * quantity;
        int discountedSubtotal;
        if (premiumCustomer == true) {
            discountedSubtotal = (int) (subtotal * 90L / 100);
        } else {
            discountedSubtotal = subtotal;
        }
        int shipping = 500;
        if (discountedSubtotal >= 5_000)
            shipping = 0;
        return discountedSubtotal + shipping;
    }
}
```

КОДОТ ПО ПРОМЕНИТЕ

```java
public class PriceCalculator {  4 usages

    public static final int MAX_UNIT_PRICE = 1_000_000;  2 usages
    public static final int MAX_QUANTITY = 100;  2 usages
    private static final int FREE_DELIVERY_THRESHOLD = 5_000;  1 usage
    private static final int SHIPPING_FEE = 500;  1 usage
    private static final long PREMIUM_CUSTOMER = 90L;  1 usage
    private static final int PERCENT_TO_DIVIDE = 100;  1 usage

    public int calculateTotal(int unitPriceCents, int quantity, boolean premiumCustomer) {  1 usage
        if (unitPriceCents < 1 || unitPriceCents > MAX_UNIT_PRICE) {
            throw new IllegalArgumentException("Unit price must be between 1 and 1000000 cents");
        }
        if (quantity < 1 || quantity > MAX_QUANTITY) {
            throw new IllegalArgumentException("Quantity must be between 1 and 100");
        }


        int subtotal = unitPriceCents * quantity;
        int discountedSubtotal;
        if (premiumCustomer == true) {
            discountedSubtotal = (int) (subtotal * PREMIUM_CUSTOMER / PERCENT_TO_DIVIDE);
        } else {
            discountedSubtotal = subtotal;
        }
        if (discountedSubtotal >= FREE_DELIVERY_THRESHOLD) {
            return discountedSubtotal;
        }
        return discountedSubtotal + SHIPPING_FEE;
    }
}
```

2. Грешки во Product.java

- Во оваа класа имаше само една грешка од типот **MagicNumber**. Поради тоа што веќе ја имав дефинирано таа константа која беше потребна и во PriceCalculator.java, само ја искористив константата од PriceCalculator.java за да нема дупликат.

КОДОТ ПРЕД ПРОМЕНАТА

```java
    if (unitPriceCents < 1 || unitPriceCents > 1_000_000) {
        throw new IllegalArgumentException("Unit price must be between 1 and 1000000 cents");
    }
```

КОДОТ ПО ПРОМЕНАТА

```
10          if (unitPriceCents < 1 || unitPriceCents > PriceCalculator.MAX_UNIT_PRICE) {
11              throw new IllegalArgumentException("Unit price must be between 1 and 1000000 cents");
12          }
```

3. Грешки во OrderService.java

- Во оваа класа имаше две грешки, едната од тип **MagicNumber**, а другата од тип **UnusedImports**.

- **Поправка**:

1. За MagicNumber грешката, поради тоа што веќе ја имав дефинирано таа константа која беше потребна и во PriceCalculator.java, само ја искористив константата од PriceCalculator.java за да нема дупликат во кодот.
2. За UnusedImports грешката, само ја избришав линијата код во која беше напишан импортот кој не е искористен.

КОДОТ ПРЕД ПРОМЕНАТА

```
1   package mk.ukim.finki.testing.shop;
2
3   import java.util.Objects;
4   import java.util.ArrayList;
5
6   public class OrderService {  no usages
```

КОДОТ ПО ПРОМЕНАТА

```
1   package mk.ukim.finki.testing.shop;
2
3   import java.util.Objects;
4
5   public class OrderService {  no usages
```

- За крај, повторно извршување на командата **./mvnw -Pstatic-analysis validate** за да провериме дали сите грешки во кодот се поправени.

```
hristinagjorgjievska@Hristinas-MacBook-Air mini-shop % ./mvnw -Pstatic-analysis validate
[INFO] Scanning for projects...
[INFO]
[INFO] ------------< mk.ukim.finki.testing:unit-testing-homework >-------------
[INFO] Building Mini Shop - Static Analysis and Unit Testing Homework 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- checkstyle:3.6.0:check (check-source-quality) @ unit-testing-homework ---
[INFO] Starting audit...
Audit done.
[INFO] You have 0 Checkstyle violations.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.360 s
[INFO] Finished at: 2026-10-26T19:56:46+01:00
[INFO] ------------------------------------------------------------------------
```

2. Unit тестови

- Unit тестови напишав за трите класи: Product.java, OrderService.java и PriceCalculator.java. За секоја од овие класи направив посебна тест класа: **ProductTest.java, OrderServiceTest.java и PriceCalculatorTest.java**.

- Тестовите ги напишав според објаснувањето во README.md, каде беа опишани и дадени одредени правила и ограничувања.

ProductTest.java

- Според правилата и органичувањата за Product.java класата, направив 4 тестови.

1. createsProductWithValidValues() - Овој тест проверува дали се креира производ со валидни вредности и дали accessor-ите code(), unitPriceCents() и availableQuantity() ги враќаат истите вредности.
- Овој тест е потребен бидејќи сите останати проверуваат дали невалидните вредности се одбиваат. Без овој тест нема да се знае дали производ со валидни вредности воопшто може да се креира.

```
19      @Test  new *
20      void createsProductWithValidValues(){
21          Product product = new Product( code: "ProductCode",  unitPriceCents: 7000,  availableQuantity: 23);
22
23          assertEquals( expected: "ProductCode", product.code());
24          assertEquals( expected: 7000, product.unitPriceCents());
25          assertEquals( expected: 23, product.availableQuantity());
26      }
```

2. rejectsBlankCode(String code) - Овој тест проверува дали производите чиј код е blank(" "), null или празен ("") се одбиваат. Според README.md, за секој производ чиј code е blank, треба да се фрли исклучок (IllegalArgumentException).
- Овој тест е параметризиран, од причина да не пишувам посебен тест за секоја невалидна вредност, напишав еден тест кој се извршува 6 пати, по еднаш за секоја вредност.
- Со @NullAndEmptySource се проверуваат null и “”.
- Со @ValueSource се проверуваат “ ”, “ ”, “\t”, “\n”.

```
33      @ParameterizedTest  new *
34      @NullAndEmptySource
35      @ValueSource(strings = {" ", "  ", "\t", "\n"})
36      void rejectsBlankCode(String code){
37          assertThrows(IllegalArgumentException.class, () -> new Product(code,  unitPriceCents: 1000,  availableQuantity: 10));
38      }
```

3. rejectsOutOfRangePrice(int price) - Овој тест проверува дали не може да се креира продукт со цена која е надвор од рангот. Според README.md, цената мора да биде од 1 до 1000000 центи (ист опсег како PriceCalculator). Вредности како 0 и 1000001 треба да фрлат IllegalArgumentException.
- Овој тест е параметризиран, со вредностите 0 и 1000001 бидејќи се надвор од дозволениот

опсег, едната е помала од минимумот, а другата е поголема од максимумот.

```
47      @ParameterizedTest  new *
48      @ValueSource(ints = {0, 1000001})
49      void rejectsOutOfRangePrice(int price){
50          assertThrows(IllegalArgumentException.class, () -> new Product( code: "ProductCode", price,  availableQuantity: 5));
51      }
```

4. rejectsNegativeValueForQuantity() - Овој тест проверува дали не може да се креира продукт кој има негативна залиха.
Според README.md, залиха мора да биде или 0 или позитивен број.

```
57      @Test  new *
58      void rejectsNegativeValueForQuantity(){
59          assertThrows(IllegalArgumentException.class, () -> new Product( code: "ProductCode",  unitPriceCents: 8500,  availableQuantity: -3));
60      }
```

PriceCalculatorTest.java

- Според правилата и органичувањата за PriceCalculator.java класата, направив 6 тестови. Бидејќи секој тест користи PriceCalculator, објектот се креира во setUp() со @BeforeEach. Така пред секој тест се креира нов калкулатор и не треба да го пишувам во секој тест посебно.

```
17      @BeforeEach  new *
18      void setUp(){
19          priceCalculator = new PriceCalculator();
20      }
```

1. rejectsOutOfRangePrice(int price) - Овој тест проверува дали се фрла исклучок (IllegalArgumentException) кога price е надвор од рангот 1 до 1000000.
- Овој тест е **параметризиран**, со вредностите 0 и 1000001 бидејќи се надвор од дозволениот опсег, едната е помала од минимумот, а другата е поголема од максимумот.

```
26      @ParameterizedTest  new *
27      @ValueSource(ints = {0, 1000001})
28      void rejectsOutOfRangePrice(int price){
29          assertThrows(IllegalArgumentException.class, () -> priceCalculator.calculateTotal(price,  quantity: 10,  premiumCustomer: true));
30      }
```

2. rejectsOutOfRangeQuantity(int quantity) - Овој тест проверува дали се фрла исклучок (IllegalArgumentException)кога quantity е надвор од рангот 1 до 100.
- Овој тест е **параметризиран**, со вредностите 0 и 101 бидејќи се надвор од дозволениот опсег, едната е помала од минимумот, а другата е поголема од максимумот.

```
36      @ParameterizedTest  new *
37      @ValueSource(ints = {0, 101})
38      void rejectsOutOfRangeQuantity(int quantity){
39          assertThrows(IllegalArgumentException.class, () -> priceCalculator.calculateTotal( unitPriceCents: 5000, quantity,  premiumCustomer: false));
40      }
```

3. calculateRegularCustomerSubtotal(int price, int quantity, int total) - Овој тест проверува дали вкупниот износ за regular customer се пресметува правилно.
- Овој тест е параметризиран со **@CsvSource,** каде секој ред е цена, количина и очекуван износ.
- Ако subtotal е < 5000, се додаваат 500 за достава.
- Ако subtotal е >= 5000, достават е 0.

- Објаснување на влезовите:

1. 350 * 10 = 3500 < 5000, се додава 500 за достава. total = 4000
2. 740 * 10 = 7400 > 5000, достава е бесплатна. total = 7400

```
61      @ParameterizedTest  new *
62      @CsvSource({
63              "350, 10, 4000",
64              "740, 10, 7400"
65      })
66      void calculateRegularCustomerSubtotal(int price, int quantity, int total){
67          assertEquals(total, priceCalculator.calculateTotal(price, quantity, premiumCustomer: false));
68      }
```

4. calculatePremiumCustomerSubtotal(int price, int quantity, int total) - Овој тест проверува дали вкупниот износ за premium customer се пресметува правилно.

- Овој тест е параметризиран со **@CsvSource**, каде секој ред е цена, количина и очекуван износ.
- Прво, од subtotal се плаќа само 90%, заокружено на помалиот цел цент.
- Ако subtotal е < 5000, се додаваат 500 за достава.

Ако subtotal е >= 5000, достава е 0.

- Објаснување на влезовите:

1. 350 * 10 = 3500 * 0.9 = 3150 < 5000, се додава 500 за достава. total = 3650
2. 740 * 10 = 7400 * 0.9 = 6660 >= 5000, достава е бесплатна. total = 6660

```
92      @ParameterizedTest  new *
93      @CsvSource({
94              "350, 10, 3650",
95              "740, 10, 6660"
96      })
97      void calculatePremiumCustomerSubtotal(int price, int quantity, int total){
98          assertEquals(total, priceCalculator.calculateTotal(price, quantity, premiumCustomer: true));
99      }
```

5. premiumCustomerSubtotalRoundedDownToWholeCents() - Овој тест проверува дали вкупниот износ кај premium корисници се заокружува на помалиот цел цент.

- Објаснување на влезот:

1759 * 0.9 = 1583,1 се заокружува на 1583 + 500 = 2083

```
107     @Test  new *
108     void premiumCustomerSubtotalRoundedDownToWholeCents(){
109         assertEquals( expected: 2083, priceCalculator.calculateTotal( unitPriceCents: 1759,  quantity: 1,  premiumCustomer: true));
110     }
```

6. shippingIsCalculatedAfterDiscount() - Овој тест проверува дали shipping-от се додава после додавање на попустот, а не пред него. Според README.md, shipping-от од 500 се додава ако износот по попустот е помал од 5000.

- Објаснување на влезот:

520 * 10 = 5200 * 0.9 = 4680 + 500 = 5180

```java
@Test  new *
void shippingIsCalculatedAfterDiscount(){
    assertEquals( expected: 5180, priceCalculator.calculateTotal( unitPriceCents: 520,  quantity: 10,  premiumCustomer: true));
}
```

OrderServiceTest.java

- Според правилата и органичувањата за OrderService.java класата, направив 11 тестови. OrderService користи ProductRepository, OrderRepository и OrderNotifier кои во проектот немаат имплементација, па затоа искористив **mock** објекти направени со **Mockito (@Mock)**.
PriceCalculator се користи вистинскиот, според барањата во README.md.
Бидејќи секој тест користи OrderService, објектот се креира во **setUp()** со **@BeforeEach**, со трите mock објекти и нов PriceCalculator. Со ова, пред секој тест се креира нов сервис и не треба да се пишува во секој тест посебно.

```java
@Mock  13 usages
private ProductRepository productRepository;
@Mock  10 usages
private OrderRepository orderRepository;
@Mock  8 usages
private OrderNotifier orderNotifier;

private OrderService orderService;  12 usages

@BeforeEach  new *
void setUp(){
    orderService = new OrderService(productRepository, orderRepository, orderNotifier, new PriceCalculator());
}
```

1. rejectsBlankCustomerId(String customerID) - Овој тест проверува дали се фрла исклучок (IllegalArgumentException) кога ID-то на купувачот е null, празно (“”) или blank. Според README.md, ID-то на купувачот не смее да биде null, празно или blank.
- Овој тест е **параметризиран**, за да не пишувам посебен тест за секоја невалидна вредност. Напишав еден тест кој се извршува 5 пати, по еднаш за секоја вредност.
- Со **@NullAndEmptySource** се проверуваат null и “”.
- Со **@ValueSource** се проверуваат “ “, “\n”, “\t”.
- Со **verifyNoInteractions** се проверува дали **сервисот не ги контактирал productRepository, orderRepository и orderNotifier**. Според README.md, невалидните податоци треба да се одбијат пред да се контактираат зависностите.

```java
@ParameterizedTest  new *
@NullAndEmptySource
@ValueSource(strings = {" ", "\n", "\t" })
void rejectsBlankCustomerId(String customerID) {
    assertThrows(IllegalArgumentException.class, () -> orderService.placeOrder(customerID,  productCode: "ProductCode",  quantity: 73,  premiumCustomer: false));

    verifyNoInteractions(productRepository, orderRepository, orderNotifier);
}
```

2. rejectsBlankProductCode(String productCode) - Овој тест проверува дали се фрла исклучок (IllegalArgumentException) кога кодот на продуктот е null, празен (“”) или blank. Според README.md, кодот на продуктот не смее да биде null, празен или blank.
- Овој тест е **параметризиран**, исто како претходниот, и се извршува 5 пати, по еднаш за секоја вредност.

- Со **@NullAndEmptySource** се проверуваат null и “”.
- Со **@ValueSource** се проверуваат “ “, “\n”, “\t”.
- Со **verifyNoInteractions** се проверува дали **сервисот не ги контактирал productRepository, orderRepository и orderNotifier**, бидејќи невалидните податоци треба да се одбијат пред да се контактираат зависностите.

```
45      @ParameterizedTest  new *
46      @NullAndEmptySource
47      @ValueSource(strings = {" ", "\n", "\t" })
48      void rejectsBlankProductCode(String productCode) {
49          assertThrows(IllegalArgumentException.class, () -> orderService.placeOrder( customerId: "CustomerID", productCode,  quantity: 73,  premiumCustomer: false));
50
51          verifyNoInteractions(productRepository, orderRepository, orderNotifier);
52      }
```

3. rejectsOutOfRangeQuantity(int quantity) - Овој тест проверува дали се фрла исклучок (IllegalArgumentException) кога количината е надвор од опсегот 1 до 100. Според README.md, количината мора да биде од 1 до 100.
Овој тест е **параметризиран**, со вредностите 0 и 101. Тие се надвор од дозволениот опсег: едната е за една помала од минимумот, а другата е за една поголема од максимумот.
Со **verifyNoInteractions** се проверува дали **сервисот не ги контактирал productRepository, orderRepository и orderNotifier**, бидејќи невалидните податоци треба да се одбијат пред да се контактираат зависностите.

```
54      @ParameterizedTest  new *
55      @ValueSource(ints = {0, 101})
56      void rejectsOutOfRangeQuantity(int quantity) {
57          assertThrows(IllegalArgumentException.class, () -> orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode", quantity,  premiumCustomer: false));
58
59          verifyNoInteractions(productRepository, orderRepository, orderNotifier);
60      }
```

4. findsProductByTheSameCode - Овој тест проверува дали продуктот се бара еднаш и со точно истиот код што е даден. Според README.md, продуктот се бара само еднаш, а кодот се користи таков каков што е даден, без бришење празни места и без менување големи и мали букви.
- Со **when/thenReturn** кажувам што да врати лажната база кога ќе се побара производот: производ со цена 1250 и залиха 80.
- Се прави нарачка од 73 парчиња. Тоа е помалку од залихата, за нарачката да помине.
- Со **verify** се проверува дали **findByCode** е повикан точно еднаш и со точно истиот код.

```
62      @Test  new *
63      void findsProductByTheSameCode() {
64          when(productRepository.findByCode("ProductCode"))
65                  .thenReturn(Optional.of(new Product( code: "ProductCode",  unitPriceCents: 1250,  availableQuantity: 80)));
66
67          orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 73,  premiumCustomer: false);
68
69          verify(productRepository).findByCode("ProductCode");
70      }
```

5. checksIfProductIsntFound - Овој тест проверува дали се фрла исклучок (ProductNotFoundException) кога продуктот не постои. Според README.md, ако продуктот не постои, треба да се фрли ProductNotFoundException.
- Со **when/thenReturn(Optional.empty())** кажувам дека лажната база нема производ со тој код.
- Со **assertThrows** се проверува дали сервисот фрла ProductNotFoundException.
- Со **verifyNoInteractions(orderRepository, orderNotifier)** се проверува дека нарачката не е зачувана и потврда не е испратена.

- **productRepository не е во verifyNoInteractions**, бидејќи сервисот треба да ја праша базата за производот. Само што таа враќа празно.

```
@Test  new *
void checksIfProductIsntFound() {
    when(productRepository.findByCode("ProductCode"))
            .thenReturn( t: Optional.empty());

    assertThrows(ProductNotFoundException.class, () -> orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 73,  premiumCustomer: false));

    verifyNoInteractions(orderRepository, orderNotifier);
}
```

6. checksIfRequestedQuantityExceedsAvailableStock - Овој тест проверува дали се фрла исклучок (InsufficientStockException) кога бараната количина е поголема од залихата на продуктот. Според README.md, ако количината е поголема од залихата, треба да се фрли InsufficientStockException.
- Со **when/then Return** кажувам дека лажната база враќа продукт со цена 1250 и залиха 72.
- Се нарачуваат 73 парчиња, т.е. само за една повеќе од залихата.
- Со **assertThrows** се проверува дали сервисот фрла InsufficientStockException.
- Со **verifyNoInteractions(orderRepository, orderNotifier)** се проверува дека нарачката не е зачувана и потврда не е испратена.

```
@Test  new *
void checksIfRequestedQuantityExceedsAvailableStock() {
    when(productRepository.findByCode("ProductCode"))
            .thenReturn(Optional.of(new Product( code: "ProductCode",  unitPriceCents: 1250,  availableQuantity: 72)));

    assertThrows(InsufficientStockException.class, () -> orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 73,  premiumCustomer: false));

    verifyNoInteractions(orderRepository, orderNotifier);
}
```

7. calculatesTotalWithPriceCalculator - Овој тест проверува дали цената на нарачката е точно пресметана. Според README.md, цената се пресметува со вистинскиот PriceCalculator.
- Со **when/then Return** кажувам дека лажната база враќа продукт со цена 1250 и залиха 79.
- Се прави нарачка од 7 парчиња за regular customer(false).
- 1250 * 7 = 8750 >= 5000, доставата е бесплатна. Вкупно 8750.
- Со **assertEquals** се проверува дали totalCents() на нарачката е 8750.

```
    @Test  new *
    void calculatesTotalWithPriceCalculator() {
        when(productRepository.findByCode("ProductCode"))
                .thenReturn(Optional.of(new Product( code: "ProductCode",  unitPriceCents: 1250,  availableQuantity: 79)));

        Order order = orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 7,  premiumCustomer: false);

        assertEquals( expected: 8750, order.totalCents());
    }
```

8. savesOrderAndSendsConfirmation - Овој тест проверува дали нарачката е зачувана и дали е испратена потврда. Според README.md, нарачката се зачувува еднаш, а потоа се праќа потврда еднаш, со истата нарачка.
- Со **when/then Return** кажувам дека лажната база враќа производ со цена 1250 и залиха 73.
- Се прави нарачка од 9 парчиња, а нарачката што ја враќа сервисот ја чувам во променливата **order**.
- Со v**erify(orderRepository).save(order)** се проверува дали нарачката е зачувана точно еднаш.
- Со **verify(orderNotifier).sendConfirmation(order)** се проверува дали потврдата е испратена точно еднаш.

- Двата verify ја користат истата променлива order, затоа тестот проверува и дека е зачувана и потврдена истата нарачка што ја враќа сервисот.

```
@Test  new *
void savesOrderAndSendsConfirmation() {
    when(productRepository.findByCode("ProductCode"))
            .thenReturn(Optional.of(new Product( code: "ProductCode",  unitPriceCents: 1250,  availableQuantity: 73)));

    Order order = orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 9,  premiumCustomer: false);

    verify(orderRepository).save(order);
    verify(orderNotifier).sendConfirmation(order);

}
```

9. returnsCorrectOrder - Овој тест проверува дали сервисот ја враќа точната нарачка. Според README.md, вратената нарачка треба да ги содржи ID-то на купувачот, кодот на производот, количината и пресметаната цена.

- Со **when/then Return** кажувам дека лажната база враќа продукт со цена 1999 и залиха 73.
- Се прави нарачка од 9 парчиња за regular customer(false).
- Очекуваната цена е 1999 * 9 = 17991 >= 5000, доставата е бесплатна. Вкупно 17991.
- Со **assertEquals** се споредува вратената нарачка со очекуваната (“CustomerID”, “ProductCode”, 9, 17991).

```
@Test  new *
void returnsCorrectOrder() {
    when(productRepository.findByCode("ProductCode"))
            .thenReturn(Optional.of(new Product( code: "ProductCode",  unitPriceCents: 1999,  availableQuantity: 73)));

    Order order = orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 9,  premiumCustomer: false);

    assertEquals(new Order( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 9,  totalCents: 17991), order);
}
```

10. doesNotSendConfirmationWhenSaveFails - Овој тест проверува што се случува кога зачувувањето на нарачката нема да успее. Според README.md, ако save фрли IllegalStateException, исклучокот треба да излезе надвор од сервисот и потврда не смее да се прати.

- Со **when/then Return** кажувам дека лажната база враќа продукт со цена 1250 и залиха 73.
- Со **doThrow** кажувам кога ќе се повика save да фрли IllegalStateException. Се пишува со doThrow наместо when, бидејќи save е void метод и не враќа ништо.
- Со **verify(orderNotifier, never())** се проверува дека потврдата не е испратена ниту еднаш. Ако нарачката не е зачувана, купувачот не треба да добие потврда.

```
@Test  new *
void doesNotSendConfirmationWhenSaveFails() {
    when(productRepository.findByCode("ProductCode"))
            .thenReturn(Optional.of(new Product( code: "ProductCode",  unitPriceCents: 1250,  availableQuantity: 73)));

    doThrow(new IllegalStateException())
            .when(orderRepository).save(any());

    assertThrows(IllegalStateException.class, () -> orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 5,  premiumCustomer: false));

    verify(orderNotifier, never()).sendConfirmation(any());
}
```

11. orderStaysSavedWhenConfirmationFails - Овој тест проверува што се случува кога праќањето на потврдата нема да успее. Според README.md, ако праќањето потврда фрли

IllegalStateException, исклучокот треба да излезе надвор од сервисот, а нарачката останува зачувана.

- Со **when/then Return** кажувам дека лажната база враќа продукт со цена 1250 и залиха 73.
- Со **doThrow** кажувам кога ќе се повика sendConfirmation да фрли IllegalStateException.
- Со **verify(orderRepository).save(any())** се проверува дека нарачката е зачувана, всушност зачувувањето не се поништува поради тоа што потврдата не успеала.



```
137     @Test  new *
138     void orderStaysSavedWhenConfirmationFails() {
139         when(productRepository.findByCode("ProductCode"))
140                 .thenReturn(Optional.of(new Product( code: "ProductCode",  unitPriceCents: 1250,  availableQuantity: 73)));
141
142         doThrow(new IllegalStateException())
143                 .when(orderNotifier).sendConfirmation(any());
144
145         assertThrows(IllegalStateException.class, () -> orderService.placeOrder( customerId: "CustomerID",  productCode: "ProductCode",  quantity: 5,  premiumCustomer: false));
146
147         verify(orderRepository).save(any());
148     }
```